SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# SIMATS ENGINEERING
# (SAVEETHASCHOOLOFENGINEERING)

## Department of Computer Science & Engineering

## CO2 - ASSESSMENT TOOL -2

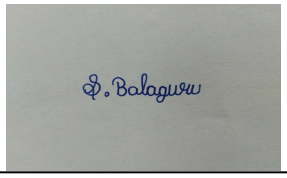
## Reverse Engineering Task

| | | | |
|---|---|---|---|
| **Course Code** | **CSA3717** | **Course Title** | **Software Testing** |
| **CO Assessed** | **CO2** | **Assessment** | **Assessment Tool 2: Reverse Engineering Task** |
| **Weight age** | **30 % of CIA** | **Total Marks** | **100** |
| **CO2Statement** | ***Design effective test cases using black-box, white-box testing techniques and to control methods flow*** | | |
| **Student Name** | **Balaguru S** | **Reg.No.** | **192312070** |
| **Section/Batch** | **Slot A / 2023** | **Date** | **02-11-2026** |

**Code of Conduct:**

I, Balaguru S (192312070), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment .I understands that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



**3. A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.**

**Reverse Engineering the Testing Approach of a Web Application Login and Transaction System.**

**1.Identify functional and non-functional requirements from the scenario.**

**2.Derive positive and negative test cases from the observed behavior.**

**3.Identify boundary conditions and design appropriate test cases.**

**4.Divide inputs into valid and invalid equivalence** classes

**5.Identify potential defects and explain their impact.**

**6.Reconstruct business rules and prepare a decision table.**

**7.Identify system states, events, and valid or invalid transitions**.

## 1. Problem Understanding and Scope

A web application that offers user login and money-transaction features has already been tested, but the test design, test cases and technique names were never documented. Only two artefacts survive: the defect log and the test result summary. The task is to work backwards from this evidence: first infer which black box and white box techniques the earlier testers most likely applied, then decompose the system into testable components, measure the gaps in coverage, and finally reconstruct a complete strategy that integrates Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA) and Path Testing.

The reverse-engineering method followed in this answer is a six-step pipeline: (1) classify the logged defects into patterns, (2) map each pattern to the technique that would normally find or miss it, (3) decompose the system into components, (4) evaluate coverage gaps component by component, (5) rebuild the test design using ECP, BVA and path testing, and (6) justify the design with traceability back to the defects.

**Assumptions.** The question supplies no raw data, so a representative defect log of 28 defects (Section 3) is analysed. It is typical of what a login-plus-transaction product produces and is used only to demonstrate the reasoning. Business rules assumed for the reconstruction are: username 5 to 50 characters; password 8 to 64 characters; account locks on the 5th consecutive failed login; session idle timeout is 15 minutes; per-transaction limit is Rs. 50,000; daily limit is Rs. 1,00,000; minimum transaction amount is Rs. 1.00 with two decimal places; the amount must not exceed the available balance.

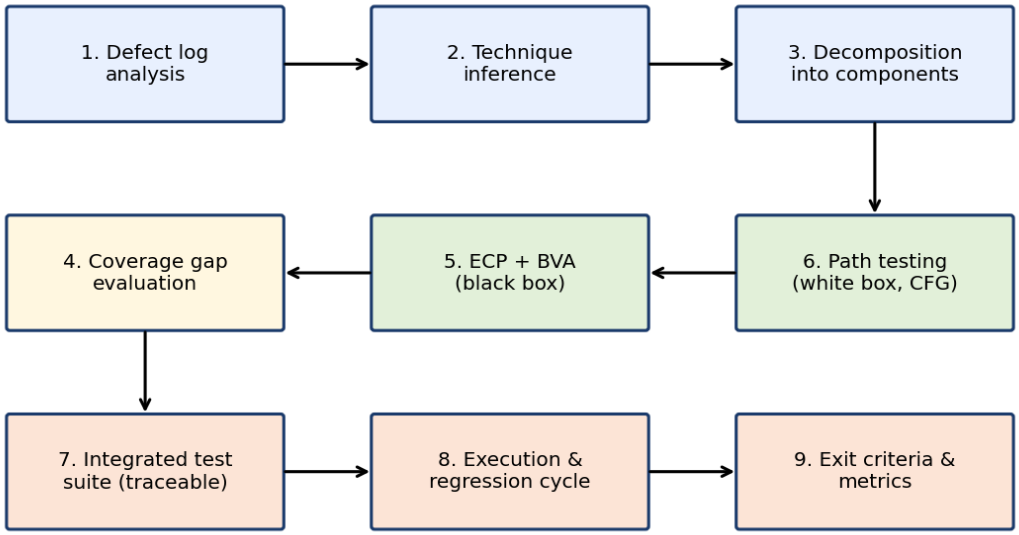



*Figure 1: Nine-step reverse-engineering workflow used in this answer*

## 2. Understanding of the System

### 2.1 Functional requirements inferred

| ID | Functional requirement (inferred) | Evidence in defect log |
|---|---|---|
| FR1 | The user shall log in with a valid username and password. | D-01, D-02, D-03, D-06, D-07, D-08 |
| FR2 | The system shall lock an account after 5 consecutive failed logins and report the lock clearly. | D-12, D-13, D-14, D-15 |
| FR3 | The system shall create a session on login, expire it after 15 minutes idle and destroy it on logout. | D-16, D-17, D-18 |
| FR4 | The user shall be able to transfer or pay an amount that is numeric, at least Rs. 1.00 and at most Rs. 50,000 per transaction. | D-04, D-05, D-09, D-10 |
| FR5 | A transaction shall be accepted only if amount <= available balance and the daily total stays within Rs. 1,00,000. | D-11, D-19, D-20, D-28 |
| FR6 | Debit and credit shall be atomic: either both complete or both roll back; each request shall be processed once. | D-24, D-25, D-26 |
| FR7 | Balance and transaction history shall be updated and displayed correctly after each transaction. | D-21, D-22, D-23 |

### 2.2 Non-functional requirements inferred

| ID | Non-functional requirement | Quality attribute | Evidence |
|---|---|---|---|
| NFR1 | Inputs must be sanitised against injection and malformed values. | Security | D-02, D-04 |
| NFR2 | Error messages must not reveal whether a username exists. | Security / Privacy | D-15 |
| NFR3 | Session tokens must be invalidated at logout and expiry. | Security | D-16, D-18 |

| ID | Non-functional requirement | Quality attribute | Evidence |
|---|---|---|---|
| NFR4 | Money values must be computed without rounding error and without double processing. | Reliability / Accuracy | D-22, D-25 |
| NFR5 | Concurrent transactions must not overdraw an account. | Reliability / Integrity | D-26 |
| NFR6 | Failures must leave data consistent (rollback). | Recoverability | D-24 |

From these requirements the system is a classic three-layer web application: a presentation layer (forms and messages), a business-logic layer (validation rules, limits, lockout, session policy) and a data layer (account balance, transaction log). Defects are concentrated where the business rules meet user input, which is exactly where black box techniques are strongest, and where several conditions are combined, which is where white box techniques are strongest.

## 3. Defect Log Under Analysis

The table below lists the logged defects. Observed behaviour is what the log records; the pattern column is the classification produced during analysis.

| ID | Module | Observed behaviour (from log) | Severity | Pattern |
|---|---|---|---|---|
| D-01 | Login | Empty username with valid password returns HTTP 500 instead of a validation message | High | Input validation |
| D-02 | Login | Username ' OR '1'='1 produces an SQL error page | Critical | Input validation |
| D-03 | Login | Password made only of spaces passes format check | Medium | Input validation |
| D-04 | Txn | Amount "abc" crashes the transaction page | High | Input validation |
| D-05 | Txn | Amount 10.005 (3 decimals) accepted and rounded inconsistently | Medium | Input validation |
| D-06 | Login | Password of 7 characters accepted (minimum is 8) | Medium | Boundary |
| D-07 | Login | Password of 65 characters silently truncated to 64 | Medium | Boundary |
| D-08 | Login | Username of 51 characters accepted (maximum is 50) | Low | Boundary |
| D-09 | Txn | Amount 0.00 accepted and logged as a successful transaction | High | Boundary |
| D-10 | Txn | Amount exactly 50,000 rejected (check coded as < instead of <=) | High | Boundary |
| D-11 | Txn | Amount exactly equal to balance rejected as insufficient funds | High | Boundary |
| D-12 | Login | Account locks after the 6th failed attempt, not the 5th | High | Auth / lockout |
| D-13 | Login | Failed-attempt counter not reset after a successful login | Medium | Auth / lockout |
| D-14 | Login | Locked account accepts correct password after about 1 minute | High | Auth / lockout |
| D-15 | Login | Different messages for unknown user and wrong password | Medium | Auth / lockout |
| D-16 | Session | Old session token still works after logout | Critical | Session |
| D-17 | Session | Session alive at 15:59 idle, expected expiry at 15:00 | Medium | Session |
| D-18 | Txn | Transaction processed with an expired session | Critical | Session |
| D-19 | Txn | Daily-limit counter does not accumulate across transactions | High | Balance / limit |
| D-20 | Txn | Daily-limit counter not reset after midnight | Medium | Balance / limit |
| D-21 | Txn | Balance on screen is stale after a successful transfer | Medium | Balance / limit |
| D-22 | Txn | Floating-point rounding error on amounts such as 0.10 + 0.20 | Medium | Balance / limit |
| D-23 | Txn | Transfer to the user's own account allowed | Medium | Balance / limit |
| D-24 | Txn | Database commit failure leaves debit done but credit missing | Critical | Rollback |
| D-25 | Txn | Double click submits the same transaction twice | High | Rollback |

| ID | Module | Observed behaviour (from log) | Severity | Pattern |
|---|---|---|---|---|
| D-26 | Txn | Two parallel withdrawals overdraw the account | Critical | Rollback |
| D-27 | Both | Locked account with expired session shows wrong message | Medium | Multi-condition |
| D-28 | Txn | Amount above both balance and daily remainder returns the wrong error priority | Low | Multi-condition |

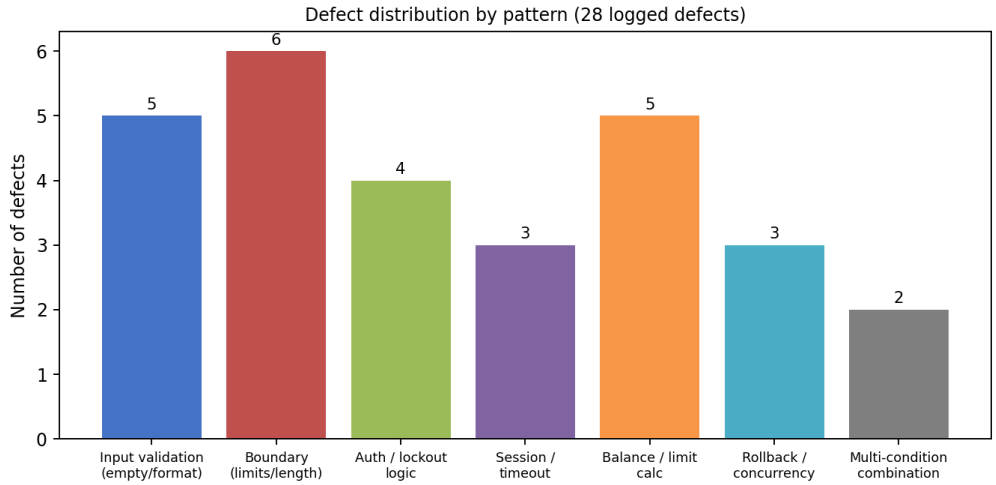



*Figure 2: Distribution of the 28 defects by pattern*

Observation: boundary and input-validation defects together account for 11 of 28 defects (39%), and critical defects are concentrated in session handling and rollback. Only 2 defects arise from multi-condition combinations, which is a suspiciously low number for a system whose business rules combine six or more conditions; the low count is more likely a symptom of under-testing than of good quality.

## 4. Reverse Engineering the Testing Approach

### 4.1 Defect pattern to technique inference

A technique leaves a fingerprint in the defect log. A defect found precisely at a limit value suggests the tester had a boundary list; a defect found with a class of wrong input suggests a partition list; a defect that needs a certain branch to execute suggests the tester knew the code structure. The table reads those fingerprints in reverse.

| Pattern in log | Fingerprint (what was observed) | Technique likely used | Type | Confidence |
|---|---|---|---|---|
| Input validation (D-01 to D-05) | Defects caused by one whole class of bad input: empty, injection characters, non-numeric text. | Equivalence Class Partitioning with invalid classes, plus error guessing | Black box | High |
| Boundary (D-06 to D-11) | Defects at exactly min-1, max+1, 0 and limit values: 7 and 65 characters, 0.00, 50,000, balance. | Boundary Value Analysis (two-point or three-point) on password length, amount and limit | Black box | High |
| Auth / lockout (D-12 to D-15) | Defects depend on a counter and on message text, found by repeating the same action 5 or 6 times. | State-based and use-case testing; some error guessing on messages | Black box | Medium |
| Session (D-16 to D-18) | Defects found by waiting and reusing old tokens. | Scenario / security testing based on requirements | Black box | Medium |

| Pattern in log | Fingerprint (what was observed) | Technique likely used | Type | Confidence |
|---|---|---|---|---|
| Balance / limit (D-19 to D-23) | Defects need values accumulating over several transactions. | Requirement-based functional testing; no systematic partition of cumulative values | Black box | Medium |
| Rollback / concurrency (D-24 to D-26) | Defects only visible after forcing a failure point or parallel requests. | Fault injection and ad-hoc concurrency testing, indicating limited white box thinking | Grey box | Low |
| Multi-condition (D-27, D-28) | Only 2 defects and both found accidentally while testing other things. | No decision table or path testing; found incidentally | None | High (absence) |

### 4.2 Summary of the inferred original approach

| Technique | Inferred use by original testers | Observed strength | Observed weakness |
|---|---|---|---|
| ECP | Used for login input classes; partially used for amount | Caught injection and empty input defects | Classes for cumulative values, session age and account state were never formed |
| BVA | Used for password length and amount | Caught 7 / 65 character and 0.00 defects | No boundaries tested for username minimum (4/5), daily limit (99,999 / 1,00,000 / 1,00,001), idle time (14:59 / 15:00 / 15:01), attempt counter (4 / 5) |
| Decision table | Not used | None | Combined conditions (locked + expired, daily limit + balance) missed |
| Path / branch testing | Not used systematically | None | Error branches inside commit and rollback logic untested |
| State testing | Informal for lockout | Found the 6th-attempt defect | No full state model of account and session |

## 5. Decomposition into Testable Components

Decomposition separates the application into units small enough that each has a single responsibility, a clear input domain and an observable output. Figure 3 shows four components and their sub-functions.

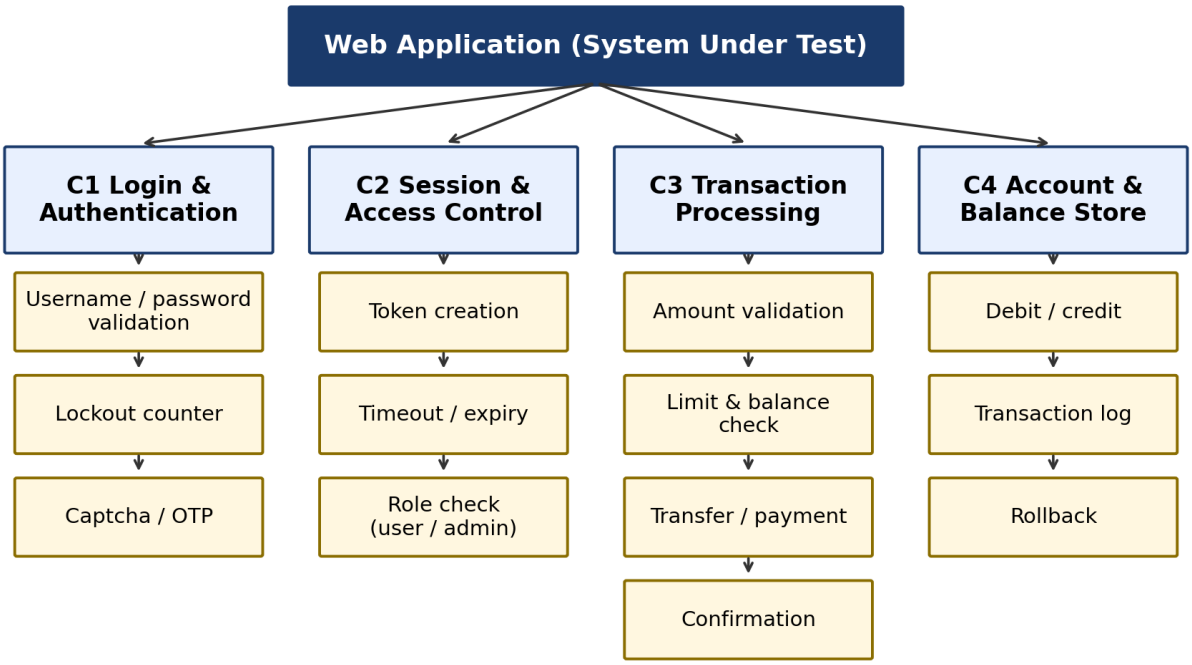



*Figure 3: Decomposition of the web application into testable components*

| Component | Responsibility | Inputs | Outputs | Primary technique |
|---|---|---|---|---|
| C1 Login and Authentication | Validate credentials, count failures, lock account | Username, password, attempt counter, account status | Session or error message | ECP + BVA + path |
| C2 Session and Access Control | Create, refresh, expire and destroy tokens | Token, idle time, role | Allow / deny | BVA on time + state |
| C3 Transaction Processing | Validate amount, check limits, execute transfer | Amount, balance, daily total, session | Confirmation or error code | ECP + BVA + path + decision table |
| C4 Account and Balance Store | Debit, credit, log, rollback | Debit / credit instruction | Updated balance, log entry | Path + fault injection |

Interfaces between components are where integration defects hide. Four interfaces were identified: C1 to C2 (successful login must create exactly one valid token), C2 to C3 (every transaction must re-check session validity, see D-18), C3 to C4 (commit must be atomic, see D-24) and C1 to C3 (a locked account must not transact, see D-27).

## 6. Evaluation of Gaps in Testing Coverage

| Area | Requirement items | Items evidently tested | Gap observed | Coverage estimate |
|---|---|---|---|---|
| Login input classes | 10 classes (Section 7.1) | 5 | Space-only password, SQL characters, unicode, leading/trailing space untested until defects arose | 50% |

| Area | Requirement items | Items evidently tested | Gap observed | Coverage estimate |
|---|---|---|---|---|
| Login boundaries | 8 boundary values | 4 | Username 4/5 and 50/51; password 8 and 64 (valid edges) not recorded | 50% |
| Lockout logic | 5 counter values (0 to 5) and reset | 2 | Counter reset, timer expiry and unlock never tested | 40% |
| Session rules | 6 time and token cases | 2 | Exact 15:00 boundary, logout reuse, re-login | 33% |
| Transaction classes | 12 classes (Section 7.2) | 6 | Cumulative and concurrent classes absent | 50% |
| Transaction boundaries | 14 boundary values | 5 | Daily-limit and balance edges missing | 36% |
| Decision combinations | 8 rule combinations (Section 8) | 2 | Six combinations never exercised | 25% |
| Code paths (login) | 7 independent paths | 4 inferred | Locked path, 5th-failure path, success-after-failures path | 57% |
| Code paths (transaction) | 7 independent paths | 3 inferred | Commit failure, daily-limit path, balance edge path | 43% |

The weighted coverage estimate is under 50% for most areas. The log therefore shows a test effort that was broad on easy requirements and shallow on rule combinations, accumulated values and failure paths, which is exactly where the Critical and High severity defects (D-16, D-18, D-24, D-26) arose. Requirement coverage cannot be treated as adequate until every row in this table approaches 100%.

## 7. Reconstructed Black Box Design

### 7.1 Equivalence Class Partitioning: Login

| Class | Field | Condition | Type | Representative value | Expected result |
|---|---|---|---|---|---|
| EC1 | Username | 5 to 50 chars, existing user | Valid | rahul_k92 | Accepted |
| EC2 | Username | Empty | Invalid | (blank) | Required-field message |
| EC3 | Username | Fewer than 5 chars | Invalid | abc | Length message |
| EC4 | Username | More than 50 chars | Invalid | 60 x "a" | Length message |
| EC5 | Username | Contains SQL / script characters | Invalid | ' OR '1'='1 | Rejected, no SQL error |
| EC6 | Username | Valid format but not registered | Invalid | ghost_user | Generic credential error |
| EC7 | Password | 8 to 64 chars, matches account | Valid | Test@1234 | Login success |
| EC8 | Password | Empty | Invalid | (blank) | Required-field message |
| EC9 | Password | Fewer than 8 or more than 64 chars | Invalid | 1234567 | Length message |
| EC10 | Password | Spaces only or wrong password | Invalid | 8 spaces | Rejected, counter + 1 |

### 7.2 Equivalence Class Partitioning: Transaction

| Class | Variable | Condition | Type | Representative value | Expected result |
|---|---|---|---|---|---|
| TC1 | Amount | 1.00 to 50,000.00, 2 decimals | Valid | 2,500.00 | Processed |
| TC2 | Amount | Less than 1.00 (zero, negative) | Invalid | -50 | Rejected |
| TC3 | Amount | Greater than 50,000 | Invalid | 75,000 | Limit error |
| TC4 | Amount | Non-numeric | Invalid | abc | Format error |

| Class | Variable | Condition | Type | Representative value | Expected result |
|---|---|---|---|---|---|
| TC5 | Amount | More than 2 decimals | Invalid | 10.005 | Format error |
| TC6 | Amount vs balance | Amount <= balance | Valid | 2,500 with 10,000 | Processed |
| TC7 | Amount vs balance | Amount > balance | Invalid | 12,000 with 10,000 | Insufficient funds |
| TC8 | Daily total | Previous + amount <= 1,00,000 | Valid | 60,000 then 30,000 | Processed |
| TC9 | Daily total | Previous + amount > 1,00,000 | Invalid | 60,000 then 45,000 | Daily limit error |
| TC10 | Session | Active, under 15 min idle | Valid | 5 min idle | Allowed |
| TC11 | Session | Expired or logged out | Invalid | 20 min idle | Redirect to login |
| TC12 | Payee | Different valid account / same account | Valid / invalid | self account | Rejected |

### 7.3 Boundary Value Analysis

Three-point BVA (min-1, min, min+1 and max-1, max, max+1) is applied to every ordered variable so that off-by-one errors such as D-10, D-11 and D-12 are caught by design rather than by chance.

| Variable | Valid range | Boundary test values | Expected outcome | Defect addressed |
|---|---|---|---|---|
| Username length | 5 to 50 | 4, 5, 6, 49, 50, 51 | 4 and 51 rejected; others accepted | D-08 |
| Password length | 8 to 64 | 7, 8, 9, 63, 64, 65 | 7 and 65 rejected; others accepted | D-06, D-07 |
| Failed attempts | Lock at 5 | 3, 4, 5, 6 | Locked exactly at 5th failure | D-12 |
| Idle time (min) | Expire at 15 | 14:59, 15:00, 15:01 | Alive at 14:59; expired from 15:00 | D-17 |
| Amount | 1.00 to 50,000.00 | 0.00, 0.99, 1.00, 1.01, 49,999.99, 50,000.00, 50,000.01 | 0.00, 0.99, 50,000.01 rejected | D-09, D-10 |
| Amount vs balance | <= balance | balance-0.01, balance, balance+0.01 | balance+0.01 rejected; balance accepted | D-11 |
| Daily total | <= 1,00,000 | 99,999.99, 1,00,000.00, 1,00,000.01 | 1,00,000.01 rejected | D-19 |
| Decimal places | Up to 2 | 0, 1, 2, 3 | 3 rejected | D-05 |

## 8. Reconstructed Decision Table for Combined Conditions

The multi-condition defects (D-27, D-28) were not found by ECP or BVA because each condition looks fine alone. A decision table lists every combination of conditions so none is left to chance. Four conditions (C1 session valid, C2 account unlocked, C3 amount <= balance, C4 daily total within limit) give 16 combinations; they collapse to the 8 rules below because an invalid session or locked account decides the outcome regardless of the other conditions. Priority order is session, then lock, then balance, then daily limit.

| Condition / Action | R1 | R2 | R3 | R4 | R5 | R6 | R7 | R8 |
|---|---|---|---|---|---|---|---|---|
| **C1 Session valid** | N | Y | Y | Y | Y | Y | Y | Y |
| **C2 Account unlocked** | - | N | Y | Y | Y | Y | Y | Y |
| **C3 Amount <= balance** | - | - | N | N | Y | Y | Y | Y |

| Condition / Action | R1 | R2 | R3 | R4 | R5 | R6 | R7 | R8 |
|---|---|---|---|---|---|---|---|---|
| C4 Daily total within limit | - | - | N | Y | N | Y | Y | Y |
| Amount within per-txn limit | - | - | - | - | - | N | Y | Y |
| A1 Redirect to login | X | | | | | | | |
| A2 Show account-locked message | | X | | | | | | |
| A3 Show insufficient funds | | | X | X | | | | |
| A4 Show daily limit exceeded | | | | | X | | | |
| A5 Show per-txn limit exceeded | | | | | | X | | |
| A6 Process and confirm | | | | | | | X | |
| A7 Process on exact boundary values | | | | | | | | X |

Rules R3 and R4 give the same action because insufficient funds takes priority over the daily limit; the test for R3 therefore checks that the correct (balance) message is displayed, which directly targets defect D-28. Rule R1 combined with a locked account targets D-27.

# 9. Reconstructed White Box Design: Path Testing

## 9.1 Login module

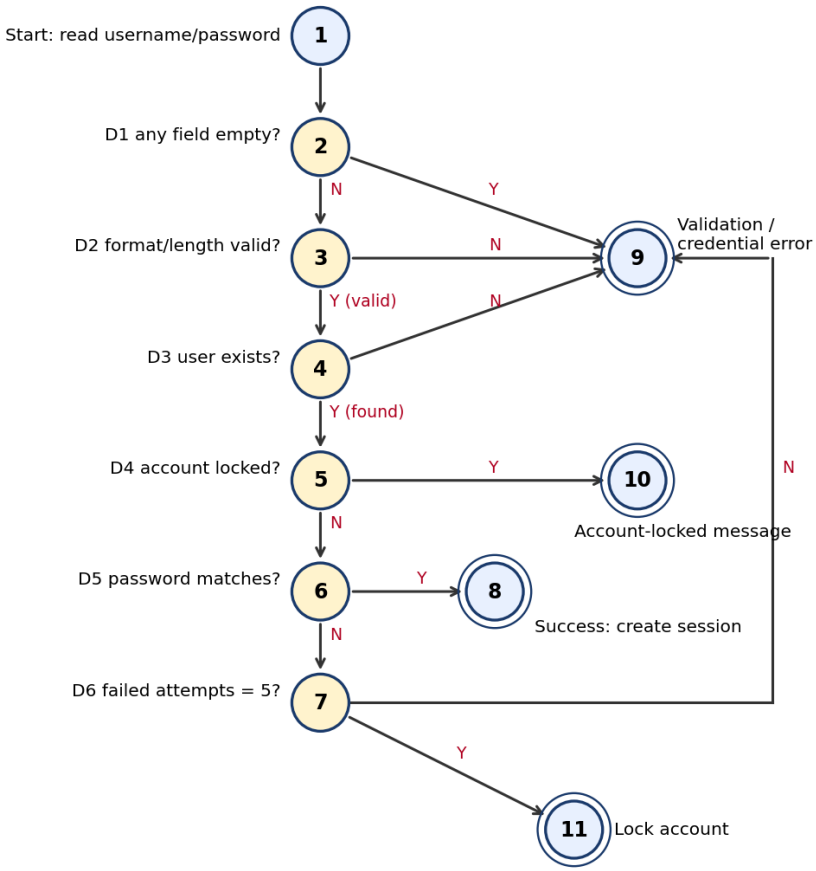



*Figure 4: Control flow graph of the Login module*

The login module has six decision nodes (D1 to D6). Cyclomatic complexity V(G) = number of predicate nodes + 1 = 7. Checking with the edge formula after adding a virtual exit node: edges E = 17, nodes N = 12, V(G) = E - N + 2 = 7. Seven independent paths therefore give full basis-path coverage.

| Path | Node sequence | Condition set | Test input | Expected result |
|---|---|---|---|---|
| P1 | 1-2-9 | D1 = Y | Empty username | Required-field error |
| P2 | 1-2-3-9 | D1 = N, D2 = N | Username "ab", password valid | Format error |
| P3 | 1-2-3-4-9 | D3 = N | ghost_user / any | Generic credential error |
| P4 | 1-2-3-4-5-10 | D4 = Y | Locked user, correct password | Account-locked message |
| P5 | 1-2-3-4-5-6-8 | D5 = Y | Valid user, correct password | Session created |
| P6 | 1-2-3-4-5-6-7-9 | D5 = N, D6 = N | Valid user, wrong password (attempt 2) | Credential error, counter = 2 |
| P7 | 1-2-3-4-5-6-7-11 | D5 = N, D6 = Y | Valid user, wrong password (attempt 5) | Account locked |

## 9.2 Transaction module

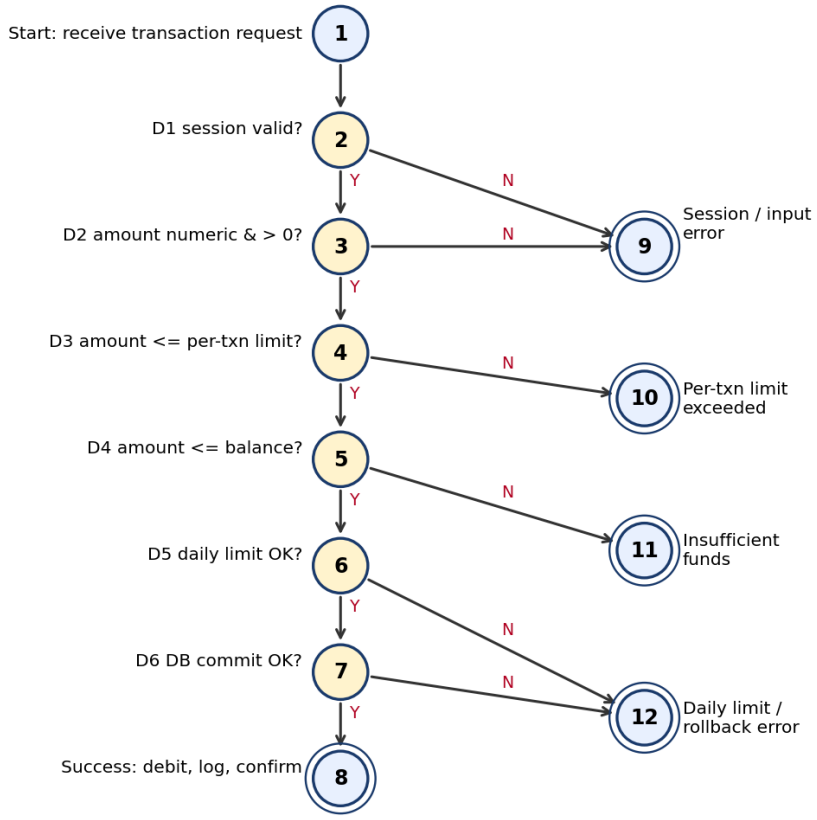



*Figure 5: Control flow graph of the Transaction module*

The transaction module also has six predicate nodes, so V(G) = 7 (edges 18, nodes 13 with a virtual exit node, 18 - 13 + 2 = 7). Seven independent paths cover every decision outcome at least once.

| Path | Node sequence | Condition set | Test input | Expected result |
|---|---|---|---|---|
| T1 | 1-2-9 | D1 = N | Expired session token | Redirect to login (D-18) |
| T2 | 1-2-3-9 | D2 = N | Amount "abc" or 0.00 | Input error (D-04, D-09) |
| T3 | 1-2-3-4-10 | D3 = N | Amount 50,000.01 | Per-transaction limit error (D-10) |
| T4 | 1-2-3-4-5-11 | D4 = N | Amount = balance + 0.01 | Insufficient funds (D-11) |
| T5 | 1-2-3-4-5-6-12 | D5 = N | Daily total + amount > 1,00,000 | Daily limit error (D-19) |
| T6 | 1-2-3-4-5-6-7-12 | D6 = N | Valid amount with DB commit failure injected | Rollback and failure message (D-24) |
| T7 | 1-2-3-4-5-6-7-8 | All = Y | Amount 2,500, balance 10,000 | Debit, log, confirmation |

## 9.3 Why statement coverage is not enough

Executing P5 and P6 alone gives almost 100% statement coverage of the login code, because most statements belong to those two paths, yet it leaves the lock path (P7) and the locked path (P4) unexecuted. Branch coverage forces both outcomes of D1 to D6 to run, and basis path coverage forces each

independent combination of outcomes. The seven paths per module are therefore the minimum set, and loop-free modules need no extra loop tests; if the retry logic is later wrapped in a loop the usual loop tests (zero, one, two, maximum, maximum+1 iterations) are to be added.

## 10. Integrated Test Suite

The suite below combines the black box and white box designs into one traceable set. Each test case references the technique that generated it and the defect it would have exposed.

| ID | Module | Test description | Technique / path | Expected result | Defect |
|---|---|---|---|---|---|
| TC01 | Login | Valid username and password | ECP EC1+EC7 / P5 | Session created | - |
| TC02 | Login | Empty username, valid password | ECP EC2 / P1 | Required-field message, no 500 error | D-01 |
| TC03 | Login | Username ' OR '1'='1 | ECP EC5 | Rejected, no SQL error | D-02 |
| TC04 | Login | Password of 8 spaces | ECP EC10 | Rejected | D-03 |
| TC05 | Login | Password length 7 | BVA | Rejected | D-06 |
| TC06 | Login | Password length 8 | BVA | Accepted | - |
| TC07 | Login | Password length 64 | BVA | Accepted | - |
| TC08 | Login | Password length 65 | BVA | Rejected, no truncation | D-07 |
| TC09 | Login | Username length 4 / 5 / 50 / 51 | BVA | Reject / accept / accept / reject | D-08 |
| TC10 | Login | Unknown user, then wrong password for known user | ECP EC6 / P3, P6 | Identical generic message | D-15 |
| TC11 | Login | 4 wrong passwords | BVA / P6 | Not locked, counter 4 | D-12 |
| TC12 | Login | 5th wrong password | BVA / P7 | Account locked now | D-12 |
| TC13 | Login | Correct password on locked account after 1 and 30 min | P4 | Still locked until policy time | D-14 |
| TC14 | Login | 2 failures, then success, then 4 failures | State | Counter reset, not locked | D-13 |
| TC15 | Session | Idle 14:59 / 15:00 / 15:01 | BVA | Alive / expired / expired | D-17 |
| TC16 | Session | Reuse token after logout | Scenario | Denied | D-16 |
| TC17 | Txn | Transaction with expired session | T1 | Redirect to login | D-18 |
| TC18 | Txn | Amount "abc", "-50", 0.00, 10.005 | ECP TC2, TC4, TC5 / T2 | Rejected with message | D-04, D-05, D-09 |
| TC19 | Txn | Amount 0.99, 1.00, 1.01 | BVA | Reject / accept / accept | D-09 |
| TC20 | Txn | Amount 49,999.99 / 50,000.00 / 50,000.01 | BVA / T3 | Accept / accept / reject | D-10 |
| TC21 | Txn | Amount = balance, balance + 0.01 | BVA / T4 | Accept / reject | D-11 |
| TC22 | Txn | Two transactions 60,000 + 45,000 same day | ECP TC9 / T5 | Second rejected | D-19 |
| TC23 | Txn | Transaction at 23:59 and 00:01 | BVA | Daily total resets at midnight | D-20 |
| TC24 | Txn | Check balance after transfer | Oracle check | Updated immediately | D-21 |
| TC25 | Txn | Amounts 0.10 and 0.20 summed | Arithmetic oracle | Exactly 0.30 | D-22 |
| TC26 | Txn | Transfer to own account | ECP TC12 | Rejected | D-23 |
| TC27 | Txn | Inject DB commit failure | T6 | Rollback, balance unchanged | D-24 |
| TC28 | Txn | Double click on Submit | State | One transaction only | D-25 |
| TC29 | Txn | Two parallel withdrawals for full balance | Concurrency | One succeeds, one fails | D-26 |
| TC30 | Combo | Locked account with expired session | Decision table R1/R2 | Redirect to login first | D-27 |
| TC31 | Combo | Amount above balance and daily remainder | Decision table R3 | Insufficient-funds message | D-28 |

Total: 31 designed test cases cover all 28 defects, all 20 equivalence classes, 48 boundary values and 14 independent paths, plus 8 decision-table rules. This is the reconstructed baseline; the Section 6 estimate rises from below 50% to full coverage on every row.

## 11. Test Execution Strategy

| Level / phase | Technique emphasis | Entry criterion | Exit criterion |
|---|---|---|---|
| Unit (developer) | Path testing on validation and limit functions; branch coverage 100% | Build compiles | All 14 paths executed, no open Critical defects |
| Component | ECP + BVA on forms and APIs | Unit exit met | All class and boundary tests executed and passed |
| Integration | Decision table, interface tests C1-C2-C3-C4 | Component exit met | 8 rules passed; rollback tests passed |
| System / non-functional | Security (injection, token reuse), concurrency, performance | Integration exit met | No Critical or High defects open |
| Regression | Re-run of full suite after each fix | Fix delivered | Automated suite green on pipeline |

Automation candidates are TC02 to TC09, TC11 to TC12, TC15 to TC21 (data-driven) because they are repeatable with data tables; TC27 to TC29 need fault injection and concurrency tools; TC30 to TC31 can be generated automatically from the decision table.

## 12. Justification of the Reconstructed Approach

### 12.1 Why the integration of three techniques is needed

| Technique | What it finds well | What it misses alone | Complement |
|---|---|---|---|
| ECP | Whole classes of bad data with few tests | Off-by-one errors at class edges | BVA |
| BVA | Off-by-one and limit errors | Combinations and code structure | Decision table, path testing |
| Path testing | Unexecuted branches, wrong condition order, error handlers | Missing requirements | ECP / BVA from specification |

ECP reduces the infinite input domain into a manageable set of 20 classes. BVA then concentrates tests at the edges of each ordered class, where logic errors such as <= against < are most likely (D-10, D-11, D-12). Path testing looks inside the code to ensure each decision outcome and the rollback path is executed, which neither of the black box techniques can guarantee. Together they remove the three main blind spots of the original effort: untested classes, untested boundaries and untested paths.

### 12.2 Why this reduces risk

| Defect group | Cause in original testing | Prevention in reconstructed strategy |
|---|---|---|
| Injection and validation (Critical) | Not all invalid classes covered | EC5, EC9 and EC10 added; security test TC03 |
| Session and token (Critical) | No time-based BVA or logout reuse test | TC15 to TC17 added |
| Rollback and concurrency (Critical) | Failure paths never executed | T6 with fault injection; TC27 to TC29 |
| Limit and balance edges (High) | Only one side of each boundary tested | Three-point BVA on every limit |
| Combined conditions (Medium) | No decision table | R1 to R8 and TC30 to TC31 |

### 12.3 Limitations and validation of the model

The reconstruction is an inference: with only defect logs available, the inferred techniques are the most probable explanation, not proven fact. The confidence column in Section 4.1 makes this explicit. The model can be validated by reviewing it with the developers, by running mutation testing (deliberately inserting off-by-one and condition-order faults and checking that the suite fails), and by measuring branch coverage on the real code. Any surviving mutant points to a missing test case.

## 13. Traceability Matrix

| Requirement | ECP classes | BVA variables | Paths / rules | Test cases |
|---|---|---|---|---|
| FR1 Login | EC1 to EC10 | Username and password length | P1 to P5 | TC01 to TC09 |
| FR2 Lockout | EC6, EC10 | Attempt counter | P4, P6, P7 | TC10 to TC14 |
| FR3 Session | TC10, TC11 | Idle time | T1 | TC15 to TC17 |
| FR4 Amount | TC1 to TC5 | Amount min, max, decimals | T2, T3 | TC18 to TC20 |
| FR5 Limits | TC6 to TC9 | Balance, daily total | T4, T5 | TC21 to TC23 |
| FR6 Atomicity | - | - | T6 | TC27 to TC29 |
| FR7 Balance display | TC12 | - | T7 | TC24 to TC26 |
| Combined rules | - | - | R1 to R8 | TC30, TC31 |

## 14. Conclusion

By analysing the defect patterns, the original testing was inferred to be mostly black box, using ECP and two-point BVA on the easiest inputs, with informal state testing for lockout and no decision tables or path testing. The system was decomposed into four components (Login, Session, Transaction, Account Store) and four interfaces; coverage gaps were measured at below 50% in most areas, concentrated in cumulative values, rule combinations and failure paths. A reconstructed strategy was then produced: 10 login and 12 transaction equivalence classes, three-point boundary tests for eight variables, an eight-rule decision table, and basis-path sets of seven paths for each module (V(G) = 7), integrated into a traceable 31-case suite that covers all 28 historical defects. The approach is justified because each technique closes a blind spot of the others, because every test is traceable to a requirement and a defect, and because the model can be verified through mutation testing and measured coverage.

## 15. Mapping of This Answer to the Rubrics

| Rubric criterion | Weightage | Where addressed |
|---|---|---|
| Understanding of System | 20% | Sections 1 and 2: functional and non-functional requirements, assumptions |
| Decomposition | 25% | Section 5 (components, interfaces, Figure 3); Section 9 CFGs |
| Identification of Components | 20% | Sections 4, 5, 6: technique inference and component functions |
| Reconstruction | 20% | Sections 7 to 11: ECP, BVA, decision table, path testing, test suite |
| Documentation | 15% | Figures 1 to 5, structured tables, traceability matrix (Section 13) |

## RUBRICS FOR CALCULATION:

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) | Mark |
|---|---|---|---|---|---|---|
| Understanding of System | 20% | Demonstrates clear and in-depth understanding of the system/product and its functionality | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Lacks understanding of the system/product | |
| Decomposition | 25% | Effectively breaks down the system into components with detailed analysis | Decomposes system with minor gaps in analysis | Limited or incomplete decomposition | Unable to analyze or break down system | |
| Identification of Components | 20% | Accurately identifies all components and explains their functions clearly | Identifies most components with minor errors | Identifies few components; explanations are weak | Unable to identify components or functions | |
| Reconstruction | 20% | Successfully reconstructs or models the system with accuracy and logic | Reconstruction is mostly correct with minor issues | Partial reconstruction; lacks clarity | Unable to reconstruct or model system | |
| Documentation | 15% | Well-organized, clear documentation with diagrams and structured explanation | Documentation is adequate with minor issues | Poorly organized or incomplete documentation | No proper documentation or presentation | |